

```
<!DOCTYPE html>
<html lang="fr">
<link rel="manifest" href="manifest.json">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="default">
<meta name="apple-mobile-web-app-title" content="Belote 1000">
<link rel="apple-touch-icon" href="https://cdn-icons-png.flaticon.com/512/3813/3813694.png">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Compteur de Belote - Édition 1000 Points</title>
  <style>
    :root {
      --primary: #2b4c7e;
      --accent: #d9534f;
      --bg: #f5f7fa;
      --card-bg: #ffffff;
      --text: #333;
      --gray: #718096;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
      background-color: var(--bg);
      color: var(--text);
      margin: 0;
      padding: 15px;
      display: flex;
      flex-direction: column;
      align-items: center;
    }

    .container {
      width: 100%;
      max-width: 500px;
      background: var(--card-bg);
      border-radius: 12px;
      box-shadow: 0 4px 15px rgba(0,0,0,0.08);
      padding: 20px;
      box-sizing: border-box;
    }
```

```
h1 {  
  text-align: center;  
  color: var(--primary);  
  margin-top: 0;  
  font-size: 24px;  
  margin-bottom: 20px;  
}
```

```
.section-box {  
  background: #fcfcfc;  
  border: 1px solid #e2e8f0;  
  border-radius: 8px;  
  padding: 15px;  
  margin-bottom: 15px;  
}
```

```
.section-box h3 {  
  margin-top: 0;  
  margin-bottom: 12px;  
  font-size: 15px;  
  color: #4a5568;  
  text-align: center;  
  text-transform: uppercase;  
  letter-spacing: 0.5px;  
}
```

```
.players-grid {  
  display: grid;  
  grid-template-columns: 1fr 10px 1fr;  
  gap: 10px;  
  align-items: center;  
}
```

```
.players-grid input {  
  padding: 8px;  
  font-size: 14px;  
  border: 1px solid #cbd5e0;  
  border-radius: 4px;  
  text-align: center;
```

```
}
```

```
.dealer-selector {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 8px;  
  margin-top: 5px;  
}
```

```
.btn-dealer {  
  background: #edf2f7;  
  border: 2px solid #edf2f7;  
  padding: 8px 4px;  
  font-size: 13px;  
  font-weight: 600;  
  border-radius: 6px;  
  cursor: pointer;  
  color: #4a5568;  
  transition: all 0.2s;  
  text-overflow: ellipsis;  
  white-space: nowrap;  
  overflow: hidden;  
}
```

```
.btn-dealer.active {  
  background: #e2efff;  
  border-color: var(--primary);  
  color: var(--primary);  
}
```

```
.score-board {  
  display: flex;  
  justify-content: space-between;  
  background: #edf2f7;  
  border-radius: 8px;  
  padding: 15px;  
  margin-bottom: 15px;  
  text-align: center;  
}
```

```
.team-score { flex: 1; }
```

```
.team-score:first-child { border-right: 2px solid #cbd5e0; }
```

```
.team-name { font-weight: bold; font-size: 14px; color: #4a5568; }
```

```
.total-points { font-size: 32px; font-weight: 900; color: var(--primary); margin-top: 5px; }
```

```
.input-grid {
  display: flex;
  gap: 15px;
  align-items: center;
  margin-bottom: 15px;
}
.input-group {
  flex: 1;
  display: flex;
  flex-direction: column;
}
.input-group label { font-size: 12px; font-weight: bold; margin-bottom: 5px; text-align: center; }
```

```
input[type="number"] {
  padding: 10px;
  font-size: 18px;
  border: 2px solid #cbd5e0;
  border-radius: 6px;
  outline: none;
  text-align: center;
}
input[type="number"]:focus { border-color: var(--primary); }
```

```
.bonus-container {
  display: flex;
  justify-content: space-between;
  gap: 15px;
  margin-bottom: 15px;
}
.bonus-column {
  flex: 1;
  background: #fff;
  padding: 10px;
  border-radius: 6px;
  border: 1px solid #e2e8f0;
  display: flex;
  flex-direction: column;
  gap: 8px;
}
```

```
.bonus-column label { font-size: 13px; display: flex; align-items: center; gap: 8px; cursor: pointer; }
```

```
.btn-validate {  
  width: 100%;  
  background: var(--primary);  
  color: white;  
  border: none;  
  padding: 12px;  
  font-size: 16px;  
  font-weight: bold;  
  border-radius: 6px;  
  cursor: pointer;  
}  
.btn-validate:hover { background: #1d3557; }
```

```
.history-section h4 { margin: 0 0 10px 0; color: var(--gray); font-size: 14px; border-bottom: 1px solid #e2e8f0; padding-bottom: 5px; }  
.history-list { max-height: 180px; overflow-y: auto; }  
.history-item { display: flex; justify-content: space-between; align-items: center; padding: 8px 10px; border-bottom: 1px solid #edf2f7; font-size: 14px; }  
.history-item:nth-child(even) { background: #f7fafc; }  
.dealer-badge { font-size: 11px; background: #cbd5e0; color: #2d3748; padding: 2px 6px; border-radius: 4px; font-weight: bold; }
```

```
.btn-reset {  
  width: 100%;  
  background: none;  
  color: var(--accent);  
  border: 1px solid var(--accent);  
  margin-top: 15px;  
  padding: 8px;  
  font-size: 14px;  
  border-radius: 6px;  
  cursor: pointer;  
}  
.btn-reset:hover { background: #fff5f5; }
```

```
/* Écran de célébration (Feu d'artifice) */
```

```
#victoryOverlay {
  display: none;
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(10, 15, 30, 0.9);
  z-index: 1000;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  color: white;
  text-align: center;
}
```

```
#fireworksCanvas {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  z-index: 1001;
  pointer-events: none;
}
```

```
.victory-content {
  position: relative;
  z-index: 1002;
  padding: 20px;
}
```

```
.victory-title {
  font-size: 36px;
  font-weight: bold;
  color: #ffd700;
  text-shadow: 0 0 10px rgba(255,215,0,0.5);
  margin-bottom: 10px;
  animation: pulse 1.5s infinite;
}
```

```
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.05); }
  100% { transform: scale(1); }
}
```

```
.btn-new-game {
  background: #ffd700;
  color: #111;
  border: none;
  padding: 15px 30px;
  font-size: 18px;
  font-weight: bold;
  border-radius: 30px;
  cursor: pointer;
  margin-top: 25px;
  box-shadow: 0 5px 15px rgba(255,215,0,0.3);
}
```

```
.btn-new-game:hover { background: #fff; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<!-- Écran du feu d'artifice caché au départ -->
```

```
<div id="victoryOverlay">
```

```
  <canvas id="fireworksCanvas"></canvas>
```

```
  <div class="victory-content">
```

```
    <div class="victory-title">🎉 VICTOIRE ! 🎉</div>
```

```
    <h2 id="winnerText" style="font-size: 24px; margin: 10px 0;">L'Équipe Gagne la partie
```

```
!</h2>
```

```
    <p id="winnerPoints" style="font-size: 18px; color: #a3b19b;"></p>
```

```
    <button class="btn-new-game" onclick="closeVictoryScreen()">Recommencer une
partie</button>
```

```
  </div>
```

```
</div>
```

```
<div class="container">
```

```
  <h1>♠ Compteur de Belote ♥</h1>
```

<!-- Configuration des équipes et joueurs -->

<div class="section-box">

<h3>Configuration de la table</h3>

<div class="players-grid">

<input type="text" id="pA1" value="Nord" placeholder="Joueur 1">

<div style="text-align:center; font-weight:bold; color:var(--gray)">VS</div>

<input type="text" id="pB1" value="Est" placeholder="Joueur 2">

<input type="text" id="pA2" value="Sud" placeholder="Joueur 3">

<div></div>

<input type="text" id="pB2" value="Ouest" placeholder="Joueur 4">

</div>

</div>

<!-- Sélection du Donneur -->

<div class="section-box" style="background: #f7fafc;">

<h3>Qui distribue ?</h3>

<div class="dealer-selector" id="dealerSelector">

<button class="btn-dealer active" id="btnP1" onclick="selectDealer(0)">Nord</button>

<button class="btn-dealer" id="btnP2" onclick="selectDealer(1)">Est</button>

<button class="btn-dealer" id="btnP3" onclick="selectDealer(2)">Sud</button>

<button class="btn-dealer" id="btnP4" onclick="selectDealer(3)">Ouest</button>

</div>

</div>

<!-- Tableau des scores -->

<div class="score-board">

<div class="team-score">

<div class="team-name" id="labelTeamA">Équipe A (Nord/Sud)</div>

<div class="total-points" id="totalA">0</div>

</div>

<div class="team-score">

<div class="team-name" id="labelTeamB">Équipe B (Est/Ouest)</div>

<div class="total-points" id="totalB">0</div>

</div>

</div>

<!-- Saisie de la donne -->


```

<div class="input-section section-box">
  <h3>Saisie des scores</h3>
  <div class="input-grid">
    <div class="input-group">
      <label for="scoreA">Points Équipe A</label>
      <input type="number" id="scoreA" min="0" max="252" placeholder="0">
    </div>
    <div style="font-weight: bold; color: #a0aec0; margin-top: 18px;">OU</div>
    <div class="input-group">
      <label for="scoreB">Points Équipe B</label>
      <input type="number" id="scoreB" min="0" max="252" placeholder="0">
    </div>
  </div>

  <div class="bonus-container">
    <div class="bonus-column">
      <label><input type="checkbox" id="beloteA"> Belote (+20)</label>
      <label><input type="checkbox" id="capotA"> <strong>Capot</strong></label>
    </div>
    <div class="bonus-column">
      <label><input type="checkbox" id="beloteB"> Belote (+20)</label>
      <label><input type="checkbox" id="capotB"> <strong>Capot</strong></label>
    </div>
  </div>

  <button class="btn-validate" id="btnAddPoints">Valider la donne</button>
</div>

<!-- Historique -->
<div class="history-section">
  <h4>Historique des donnees</h4>
  <div id="historyList" class="history-list"></div>
</div>

  <button class="btn-reset" id="btnReset">Recommencer la partie</button>
</div>

<script>

```

```
let totalScoreA = 0;
let totalScoreB = 0;
const BASE_POINTS = 162;
const CAPOT_POINTS = 252;
const GOAL_POINTS = 1000; // Seuil du feu d'artifice
```

```
let currentDealerIndex = 0;
let currentDealerName = "Nord";
```

```
const inputA = document.getElementById('scoreA');
const inputB = document.getElementById('scoreB');
const beloteA = document.getElementById('beloteA');
const beloteB = document.getElementById('beloteB');
const capotA = document.getElementById('capotA');
const capotB = document.getElementById('capotB');
const btnAdd = document.getElementById('btnAddPoints');
const btnReset = document.getElementById('btnReset');
const displayTotalA = document.getElementById('totalA');
const displayTotalB = document.getElementById('totalB');
const historyList = document.getElementById('historyList');
```

```
const inputsJoueurs = [
  document.getElementById('pA1'),
  document.getElementById('pB1'),
  document.getElementById('pA2'),
  document.getElementById('pB2')
];
```

```
const boutonsDonneurs = [
  document.getElementById('btnP1'),
  document.getElementById('btnP2'),
  document.getElementById('btnP3'),
  document.getElementById('btnP4')
];
```

```
inputsJoueurs.forEach((input, index) => {
  input.addEventListener('input', () => {
    boutonsDonneurs[index].textContent = input.value || `Joueur ${index+1}`;
    if(index === currentDealerIndex) {
```

```

        currentDealerName = input.value || `Joueur ${index+1}`;
    }
    });
});

```

```

window.selectDealer = function(index) {
    currentDealerIndex = index;
    boutonsDonneurs.forEach(btn => btn.classList.remove('active'));
    boutonsDonneurs[index].classList.add('active');
    currentDealerName = inputsJoueurs[index].value || `Joueur ${index+1}`;
}

```

```

function passerAuDonneurSuivant() {
    let nextIndex = (currentDealerIndex + 1) % 4;
    selectDealer(nextIndex);
}

```

```

// --- Saisie automatique miroir ---
inputA.addEventListener('input', () => {
    if (capotA.checked || capotB.checked) return;
    if (inputA.value === "") { inputB.value = ""; return; }
    let valA = parseInt(inputA.value) || 0;
    if (valA > BASE_POINTS) { valA = BASE_POINTS; inputA.value = BASE_POINTS; }
    if (valA < 0) { valA = 0; inputA.value = 0; }
    inputB.value = BASE_POINTS - valA;
});

```

```

inputB.addEventListener('input', () => {
    if (capotA.checked || capotB.checked) return;
    if (inputB.value === "") { inputA.value = ""; return; }
    let valB = parseInt(inputB.value) || 0;
    if (valB > BASE_POINTS) { valB = BASE_POINTS; inputB.value = BASE_POINTS; }
    if (valB < 0) { valB = 0; inputB.value = 0; }
    inputA.value = BASE_POINTS - valB;
});

```

```

// --- Verrous logiques Capot (Fixés à 252) ---
capotA.addEventListener('change', () => {

```

```

    if (capotA.checked) {
        capotB.checked = false;
        inputA.value = CAPOT_POINTS;
        inputB.value = 0;
        inputA.disabled = true;
        inputB.disabled = true;
    } else {
        libererInputs();
    }
});

```

```

capotB.addEventListener('change', () => {
    if (capotB.checked) {
        capotA.checked = false;
        inputB.value = CAPOT_POINTS;
        inputA.value = 0;
        inputA.disabled = true;
        inputB.disabled = true;
    } else {
        libererInputs();
    }
});

```

```

function libererInputs() {
    inputA.disabled = false;
    inputB.disabled = false;
    inputA.value = "";
    inputB.value = "";
}

```

```

// --- Validation d'une donne ---
btnAdd.addEventListener('click', () => {
    if (inputA.value === "" && inputB.value === "" && !capotA.checked && !capotB.checked)
return;

```

```

    let donneA = parseInt(inputA.value) || 0;
    let donneB = parseInt(inputB.value) || 0;
    let detailA = "";
    let detailB = "";

```

```

if (capotA.checked || donneA === BASE_POINTS) {
    donneA = CAPOT_POINTS;
    donneB = 0;
    detailA += " <b>(Capot)</b>";
} else if (capotB.checked || donneB === BASE_POINTS) {
    donneB = CAPOT_POINTS;
    donneA = 0;
    detailB += " <b>(Capot)</b>";
}

```

```

if (beloteA.checked) { donneA += 20; detailA += " <small>(Belote)</small>"; }
if (beloteB.checked) { donneB += 20; detailB += " <small>(Belote)</small>"; }

```

```

totalScoreA += donneA;
totalScoreB += donneB;
displayTotalA.textContent = totalScoreA;
displayTotalB.textContent = totalScoreB;

```

```

const historyItem = document.createElement('div');
historyItem.className = 'history-item';
historyItem.innerHTML = `
    <span style="flex:1; text-align:left;">+ ${donneA}${detailA}</span>
    <span class="dealer-badge">Donneur: ${currentDealerName}</span>
    <span style="flex:1; text-align:right;">+ ${donneB}${detailB}</span>
`;
historyList.insertBefore(historyItem, historyList.firstChild);

```

```

libererInputs();
capotA.checked = false;
capotB.checked = false;
beloteA.checked = false;
beloteB.checked = false;
passerAuDonneurSuivant();

```

```

// Vérification des 1000 points
checkVictory();

```

```
});
```

```
// --- Déclencheur de Victoire ---
```

```
function checkVictory() {  
  if (totalScoreA >= GOAL_POINTS || totalScoreB >= GOAL_POINTS) {  
    let winnerName = "";  
    let finalScores = "";  
  
    if (totalScoreA >= GOAL_POINTS && totalScoreA > totalScoreB) {  
      const n1 = inputsJoueurs[0].value || "Nord";  
      const n2 = inputsJoueurs[2].value || "Sud";  
      winnerName = `L'Équipe A (${n1} & ${n2})`;  
      finalScores = `${totalScoreA} pts à ${totalScoreB}`;  
    } else {  
      const e1 = inputsJoueurs[1].value || "Est";  
      const e2 = inputsJoueurs[3].value || "Ouest";  
      winnerName = `L'Équipe B (${e1} & ${e2})`;  
      finalScores = `${totalScoreB} pts à ${totalScoreA}`;  
    }  
  }  
}
```

```
document.getElementById('winnerText').textContent = `${winnerName} remporte la  
partie !`;
```

```
document.getElementById('winnerPoints').textContent = `Score final : ${finalScores}`;
```

```
// Déployer l'overlay et lancer l'animation
```

```
document.getElementById('victoryOverlay').style.display = 'flex';  
startFireworks();
```

```
  }  
}
```

```
window.closeVictoryScreen = function() {  
  document.getElementById('victoryOverlay').style.display = 'none';  
  stopFireworks();  
  recommencerPartie(false);  
}
```

```
function recommencerPartie(demanderConfirmation = true) {  
  if (demanderConfirmation && !confirm("Effacer la partie en cours ?")) return;  
  totalScoreA = 0;
```

```

totalScoreB = 0;
displayTotalA.textContent = "0";
displayTotalB.textContent = "0";
historyList.innerHTML = "";
libererInputs();
capotA.checked = false;
capotB.checked = false;
beloteA.checked = false;
beloteB.checked = false;
selectDealer(0);
}

```

```

btnReset.addEventListener('click', () => recommencerPartie(true));

```

```

// =====
// MOTEUR DU FEU D'ARTIFICE (CANVAS HTML5)
// =====
const canvas = document.getElementById('fireworksCanvas');
const ctx = canvas.getContext('2d');
let fireworksAnimationId = null;
let particles = [];
let fireworkLaunchTimer = 0;

```

```

function resizeCanvas() {
    canvas.width = window.innerWidth;
    canvas.height = window.innerHeight;
}
window.addEventListener('resize', resizeCanvas);

```

```

class Particle {
    constructor(x, y, color) {
        this.x = x;
        this.y = y;
        this.color = color;
        this.radius = Math.random() * 3 + 1;
        const angle = Math.random() * Math.PI * 2;
        const speed = Math.random() * 6 + 2;
    }
}

```

```

    this.velocityX = Math.cos(angle) * speed;
    this.velocityY = Math.sin(angle) * speed;
    this.alpha = 1;
    this.decay = Math.random() * 0.015 + 0.01;
  }
  draw() {
    ctx.save();
    ctx.globalAlpha = this.alpha;
    ctx.beginPath();
    ctx.arc(this.x, this.y, this.radius, 0, Math.PI * 2);
    ctx.fillStyle = this.color;
    ctx.fill();
    ctx.restore();
  }
  update() {
    this.x += this.velocityX;
    this.y += this.velocityY;
    this.velocityY += 0.05; // Force de gravité
    this.alpha -= this.decay;
  }
}

```

```

function createExplosion(x, y) {
  const colors = ['#ffd700', '#ff3333', '#33ff33', '#3333ff', '#ff33ff', '#33ffff', '#ffffff'];
  const randomColor = colors[Math.floor(Math.random() * colors.length)];
  for (let i = 0; i < 60; i++) {
    particles.push(new Particle(x, y, randomColor));
  }
}

```

```

function animateFireworks() {
  ctx.fillStyle = 'rgba(10, 15, 30, 0.2)'; // Effet de traînée
  ctx.fillRect(0, 0, canvas.width, canvas.height);

```

```

  // Gestion du déclenchement cyclique des fusées
  fireworkLaunchTimer++;
  if (fireworkLaunchTimer % 25 === 0) {
    const randomX = Math.random() * (canvas.width - 200) + 100;
    const randomY = Math.random() * (canvas.height - 300) + 100;
    createExplosion(randomX, randomY);
  }

```



```
}
```

```
// Gestion des débris
```

```
for (let i = particles.length - 1; i >= 0; i--) {  
  particles[i].update();  
  particles[i].draw();  
  if (particles[i].alpha <= 0) {  
    particles.splice(i, 1);  
  }  
}
```

```
fireworksAnimationId = requestAnimationFrame(animateFireworks);  
}
```

```
function startFireworks() {  
  resizeCanvas();  
  particles = [];  
  fireworkLaunchTimer = 0;  
  animateFireworks();  
}
```

```
function stopFireworks() {  
  cancelAnimationFrame(fireworksAnimationId);  
  ctx.clearRect(0, 0, canvas.width, canvas.height);  
}
```

```
</script>
```

```
</body>
```

```
</html>
```